

Нов български университет
Департамент „Информатика“

КУРСОВА РАБОТА

Форум с автоматично модериране на груби коментари

Проект ForumGuard — анализ на сентимент с ML.NET и NAS-BERT

Дисциплина: **CSCB634 — Практика по програмиране и бази данни**

Изготвили (екип от трима студенти):

Радослав Гурев — F112656

Алиса Спасова — F112658

Любослав Станков — F114428

Семестър: 6

Преподавател: доц. Лъчезар Томов

Дата: 22.06.2026 г.

Съдържание

1. Условие на задачата
2. Въведение и предметна област
3. Преглед на сродни решения
4. Архитектура на решението
5. Домейн модел и жизнен цикъл на коментара
6. Поток на модериране (Chain of Responsibility)
7. Роли и оторизация
8. ML компонент — анализ на токсичност (NAS-BERT)
9. Слой за данни (EF Core + SQL Server)
10. Приложени шаблони за дизайн
11. Технологичен стек
12. Тестване
13. Стартиране и употреба
14. Как се обучава ML моделът
15. Разпределение на задачите в екипа
16. Използване на AI асистент за консултация
17. Заключение
18. Използвана литература
19. Бележка относно изходния код

1. Условие на задачата

Условието на задачата е цитирано дословно по-долу:

„Трябва да се създаде форум, в който всеки въведен коментар бива анализиран за „сентимент“, т.е. за емоции. Технологията е ML.NET 2.0, сайтът трябва да е на NET.Core. Трябва да има потребители, модератори и администратори. Всеки написан коментар трябва автоматично да бъде анализиран с помощта на предварително тренирания върху писани от вас коментари модел, в този пример, с алгоритъма NAS-BERT. Флагнатите за грубо съдържание коментари трябва да се подават на модератор, т.е. да излизат в таблица в страница, в която той или тя да преценяват дали да одобрят даден коментар, или не. Ако коментарът мине анализа успешно и не е флагнат, той трябва да се появява автоматично. Администраторът дава и отнема права на модераторите, но няма права върху коментарите. Той обаче може да активира и деактивира профилите на потребителите.“

1.1. Кратко тълкуване на условието

От условието излизат следните функционални изисквания, около които е построен целият проект:

- **Автоматичен анализ на всеки коментар.** Преди да стане публично видим, всеки коментар минава през предварително обучен класификатор, който решава дали текстът е приемлив („Clean“) или груб/токсичен („Toxic“).
- **Два изхода от анализа.** Коментар, който мине анализа чисто, се публикува автоматично; флагнат коментар се скрива и отива в опашка за ръчен преглед от модератор.
- **Три роли с ясно разделени права.** Потребител (пише коментари), модератор (одобрява или отхвърля флагнати коментари) и администратор (управлява правата на модераторите и активността на профилите).
- **Строго разделение на отговорностите.** Администраторът няма власт върху коментарите — не може да модерира; модераторът пък не управлява акаунти и роли.
- **Обучение върху собствени коментари.** Моделът се обучава офлайн с алгоритъма NAS-BERT върху ръчно написан набор от коментари.

Бележка за версиите на технологиите. Условието посочва „ML.NET 2.0“ и „NET.Core“. Проектът е написан на **.NET 8 LTS** с **ML.NET 4.x**. Точно ML.NET 2.0 въведе API-то за класификация на текст с NAS-BERT (`TextClassification`), което се ползва тук; същият API върви без промяна и в ML.NET 4.x за .NET 8. Тоест използвам по-нова версия само заради актуалност — алгоритъмът и програмният интерфейс са същите. По-подробната обосновка е в раздел 10.

2. Въведение и предметна област

ForumGuard е уеб-форум, в който модерирането на грубо съдържание става автоматично с машинно самообучение. Проектът показва как класически софтуерни архитектури (слоест монолит, проектни шаблони) се съчетават с компонент за обработка на естествен език, така че форумът да остане безопасен за хората, без някой постоянно да следи всеки коментар.

2.1. Защо е нужно модерирание на токсично съдържание

Онлайн форумите постоянно се сблъскват с обиди, тормоз и враждебни коментари. Ръчното модерирание на всичко не се мащабира: коментарите идват по-бързо, отколкото модераторите могат да ги прегледат, а докато чакат за преглед, вредното съдържание вече е стигнало до читателите. В другата крайност, ако блокираме всичко само автоматично, рискуваме грешки — фалшиви аларми, които спират напълно нормални мнения.

Затова ForumGuard върви по **среден път**: моделът сам обработва голямата „чиста“ част от трафика и пуска тези коментари веднага, а модераторите се занимават само с коментарите, които анализът е отбелязал като съмнителни. Така хората гледат там, където наистина има нужда от тях.

2.2. Защо автоматичен анализ на сентимента

Класификацията на „сентимент“ или токсичност е задача от обработката на естествения език (NLP) — на даден текст се поставя етикет от предварително зададен набор (тук два: `Clean` / `Toxic`). Съвременните решения ползват предварително обучени езикови модели от семейството на трансформърите, които хващат контекста и смисъла много по-добре от обикновените филтри по ключови думи. NAS-BERT, посочен в условието, е компактен трансформър, намерен чрез търсене на невронна архитектура (Neural Architecture Search) и оптимизиран да е по-малък и по-бърз, без да губи от качеството [1][2].

Важно решение в ForumGuard е принципът „**fail-safe**“ (безопасно при отказ): при всяка грешка в анализатора или конвейера коментарът отива за ръчен преглед, но *никога* не се публикува автоматично. Така технически проблем не може да изкара навън потенциално вредно съдържание.

3. Преглед на сродни решения

За да обясня защо избрах този подход, по-долу разглеждам три вида технологии за откриване на токсично съдържание, подредени от най-проста към най-сложна и качествена.

3.1. Филтри по ключови думи и регулярни изрази

Най-простият вариант е списък със забранени думи (понякога зададени с регулярни изрази), който маркира коментара, ако съвпадне с някоя от тях.

- **Предимства:** много лесно се прави, работи мигновено, напълно прозрачно и под контрол.
- **Недостатъци:** не разбира контекста (изпуска завоалирани обиди и маркира съвсем нормални думи), лесно се заобикаля с нарочни правописни грешки и трябва някой постоянно да поддържа списъка на ръка.

ForumGuard *ползва* този подход, но само като бърз пръв филтър (`ProfanityPreFilterHandler`), а не като единствена защита.

3.2. Облачни услуги за анализ — Google Perspective API

Perspective API на Jigsaw/Google връща оценка за „toxicity“ на текст през HTTP заявка [3].

- **Предимства:** високо качество, не се налага да обучаваш нищо сам, поддържа много езици.
- **Недостатъци:** зависиш от външна услуга и интернет, пращаш потребителски данни на трета страна (проблем за поверителността), има лимити на заявките и евентуални разходи; освен това противоречи на изискването моделът да е обучен „върху писани от вас коментари“.

3.3. Трансформър-базирани класификатори (BERT / RoBERTa / NAS-BERT)

Днешният стандарт е дообучаване (fine-tuning) на предварително обучен трансформър върху набор от примерни коментари [1][4][5].

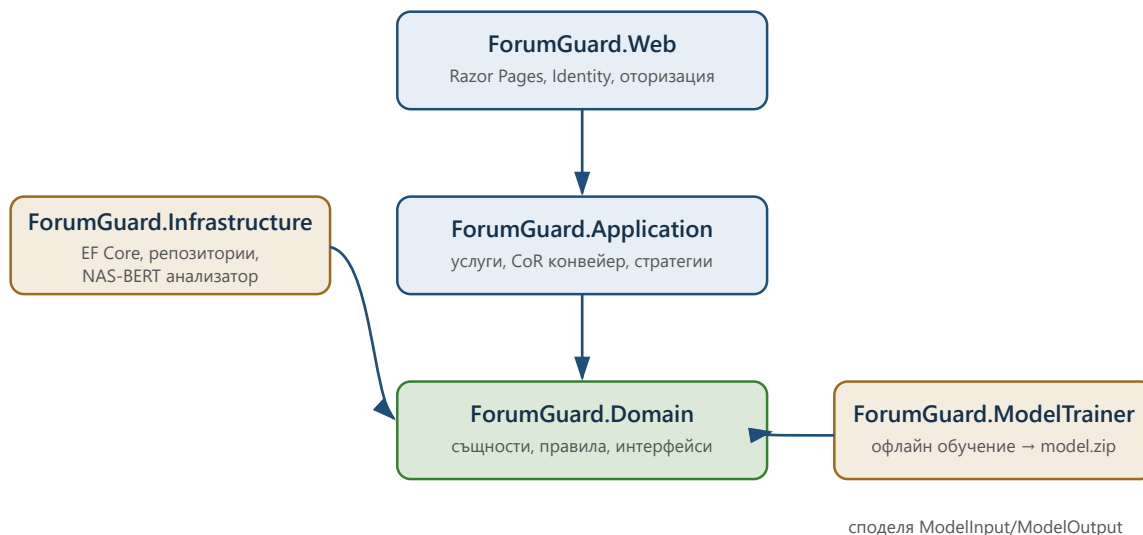
- **Предимства:** разбира контекста и смисъла, висока точност, работи изцяло локално (без външна услуга) и се учи от собствените ти данни.
- **Недостатъци:** по-сложно е за правене, трябва ти набор за обучение, а дообучаването на процесор е бавно (затова го изнасям като офлайн стъпка).

ForumGuard залага на третия подход — чрез тренера `TextClassification` на ML.NET с архитектурата **NAS-BERT (RoBERTa)** през `Microsoft.ML.TorchSharp` [2] [6] — но запазва и филтъра по ключови думи като евтин пръв слой за очевидните случаи.

4. Архитектура на решението

ForumGuard е **слоест монолит** върху ASP.NET Core Razor Pages. Кодът е разделен на пет проекта плюс един за тестове, като зависимостите сочат само навътре — към домейна. Така бизнес-логиката не зависи от инфраструктурата (база данни, ML модел, уеб рамка) и може да се тества изолирано.

Проект	Отговорност
ForumGuard .Domain	Същности (<code>Comment</code> , <code>ForumThread</code> , <code>ModerationDecision</code>), изброени типове, правилата за жизнения цикъл (<code>CommentStatusTransitions</code>), доменните интерфейси (<code>ICommentAnalyzer</code> , <code>ISpecification<T></code> , репозиторни интерфейси) и ролеви константи. Не зависи от нищо външно.
ForumGuard .Application	Случаи на употреба и услуги, конвейерът за моделиране (Chain of Responsibility) с неговите handler-и, стратегиите <code>IRoleWorkspace</code> , DTO обектите, <code>ModerationOptions</code> и <code>Result<T></code> .
ForumGuard .Infrastructure	<code>ForumGuardDbContext</code> (EF Core + SQL Server), репозитории и Unit of Work, реализации на спецификациите, <code>NasBertCommentAnalyzer</code> и <code>KeywordCommentAnalyzer</code> , EF Core миграции и <code>ApplicationUser</code> .
ForumGuard .Web	Razor Pages, ASP.NET Core Identity, оторизационни requirements и handler-и, резолвер на ролевото табло, услуги на уеб слоя, композиционен корен (<code>Program.cs</code>) и началното зареждане на данни.
ForumGuard .ModelTrainer	Конзолно приложение за офлайн обучение на NAS-BERT модела върху seed корпуса и експорт на <code>model.zip</code> .
ForumGuard .Tests	NUnit тестов проект — единични (unit) и интеграционни тестове за всички слоеве.



Стрелките сочат в посока на зависимост: всеки слой зависи единствено навътре, към домейна.

Domain не реферира нито един друг проект. DTO обектите никога не разкриват EF същности.

Фиг. 1: Слостна архитектура на ForumGuard и посоки на зависимост (Web → Application → Domain; Infrastructure → Application/Domain).

Това поддръждане следва принципа на *обърнатите зависимости*: `ForumGuard.Domain` описва интерфейсите (например `ICommentAnalyzer`, `IRepository<T>`), а `ForumGuard.Infrastructure` ги изпълнява. Така ядрото на бизнес-логиката не е вързано за конкретна база данни или ML библиотека и може да се тества с фалшиви реализации (fakes/mocks).

5. Домейн модел и жизнен цикъл на коментара

В основата на домейна стоят четири същности и три изброени типа. Те са единственият меродавен източник за структурата на данните в системата.

5.1. Същности

Същност	Ключови полета
<code>ApplicationUser</code> (наследява <code>IdentityUser<Guid></code>)	<code>DisplayName</code> , <code>IsActive</code> (управлявано от администратор, по подразбиране <code>true</code>), <code>CreatedAtUtc</code> .
<code>ForumThread</code>	<code>Id</code> (Guid PK), <code>Title</code> (макс. 200 символа), <code>CreatedById</code> , <code>CreatedAtUtc</code> .
<code>Comment</code>	<code>Id</code> , <code>ThreadId</code> , <code>AuthorId</code> , <code>Body</code> (макс. 4000 символа), <code>Status</code> , <code>CreatedAtUtc</code> , <code>PublishedAtUtc</code> (nullable), <code>AnalysisLabel</code> (nullable), <code>AnalysisScore</code> (nullable), <code>AnalyzedAtUtc</code> (nullable).
<code>ModerationDecision</code>	<code>Id</code> , <code>CommentId</code> , <code>ModeratorId</code> , <code>Decision</code> , <code>DecidedAtUtc</code> , <code>Reason</code> (nullable, макс. 500 символа).

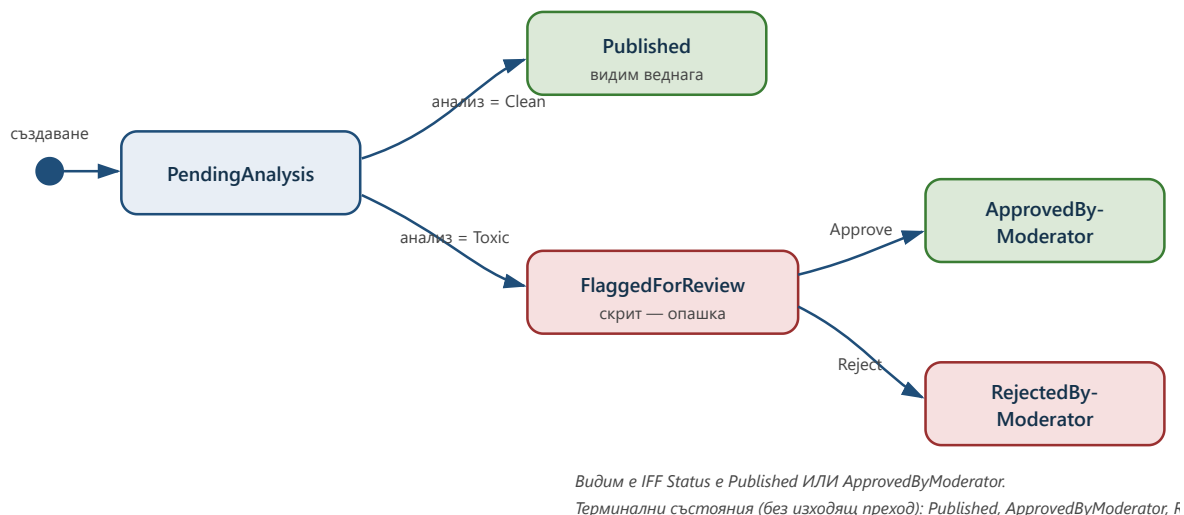
Същността `Comment` е направена *самозащитаваща се* (`sealed` клас с `private set` полета): състоянието се сменя само през методите `ApplyAnalysis`, `Approve` и `Reject`, всеки от които минава през пазач (`GuardTransition`), който хвърля `InvalidCommentTransitionException` при непозволен преход. Така откън е невъзможно състоянието да стане невалидно.

5.2. Изброени типове

- `CommentStatus` { `PendingAnalysis`, `Published`, `FlaggedForReview`, `ApprovedByModerator`, `RejectedByModerator` }
- `ToxicityLabel` { `Clean`, `Toxic` }
- `ModerationOutcome` { `Approved`, `Rejected` }

5.3. Машина на състоянията

Жизненият цикъл на коментара е детерминирана машина на състоянията. Чистата функция `CommentStatusTransitions.IsAllowed(from, to)` връща `true` точно за четирите позволени прехода и `false` за всичко останало (включително преход към самото състояние и всеки опит за връщане в `PendingAnalysis`).



Фиг. 2: Машина на състоянията на коментара. Зелено — публично видими терминални състояния; червено — скрити състояния.

5.4. Инвариант за видимост

Целостта на форума се пази от едно кратко правило (инвариант), записано в свойството `Comment.IsPubliclyVisible`:

Коментар е публично видим **тогава и само тогава**, когато статусът му е `Published` или `ApprovedByModerator`.

При всички останали състояния (`PendingAnalysis`, `FlaggedForReview`, `RejectedByModerator`) коментарът остава скрит. Домейнът също така следи `PublishedAtUtc` да е попълнено винаги, когато коментарът е видим, и да е `null` при всяко скрито състояние.

6. Поток на модерирание (Chain of Responsibility)

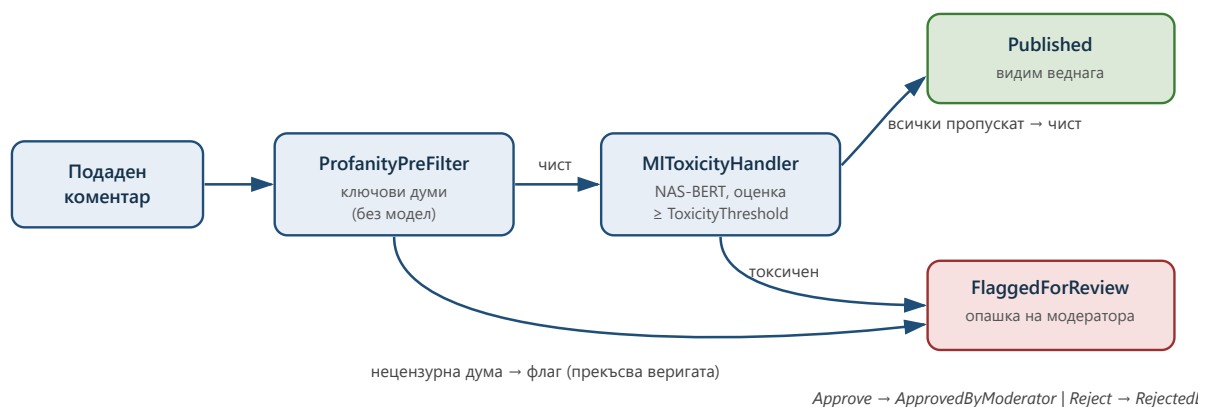
Дали коментарът е чист или токсичен се решава от **верига от отговорности** (Chain of Responsibility). Веригата е поредица от звена (handler-и) и всяко звено или *флагва* коментара (и спира веригата), или го *пуска* към следващото звено. Ако всички звена го пуснат, коментарът е чист и се публикува.

Редът на веригата е:

1. `ProfanityPreFilterHandler` (`Order = 100`) — ползва `KeywordCommentAnalyzer` (бърз, без модел). Ако види явна нецензурна дума, флагва коментара, без изобщо да вика ML модела.

2. `MlToxicityHandler` (`Order = 200`) — ползва `NAS-BERT` анализатора (`ICommentAnalyzer`). Флагва, когато оценката за токсичност е $\geq$ `ToxicityThreshold` (включително при равенство).

Веригата лесно се *разширява*: за ново звено (напр. за дължина или спам) е достатъчно да напишеш клас, който изпълнява `ICommentModerationHandler`, и да го регистрираш в DI контейнера, без да пипаш старите звена (принцип Open/Closed). Редът на изпълнение идва от свойството `Order`.



Фиг. 3: Поток на модерирание. Чист коментар се публикува автоматично; флагнат коментар постъпва в опашката, където модератор го одобрява или отхвърля.

6.1. Принцип fail-safe

Ако някое звено хвърли изключение по време на оценката, конвейерът **не** го пуска навън, а го хваща, записва го в логa (с името на звеното и идентификатора на коментара) и връща *флагнат* резултат с максимална оценка (`FailSafeScore = 1.0`). Така технически проблем (например ако моделът не успее да се зареди) винаги праща коментара за ръчен преглед и **никога** не води до автоматично публикуване. Единственото изключение, което минава нормално навън, е `OperationCanceledException` при отказана заявка — тогава просто няма решение за записване.

Резултатът от конвейера е обектът `ModerationVerdict { Label, Score, IsFlagged, FlaggingHandlerName }`, който услугата за подаване на коментар (`CommentSubmissionService`) превръща в съответния `CommentStatus` и записва в базата.

7. Роли и оторизация

Влизането и правата минават през **ASP.NET Core Identity**. Дефинирани са точно три роли чрез константи в `ForumRoles` : `"User"` , `"Moderator"` и `"Administrator"` .

Роля	Правомощия
User	Създава теми и пише коментари.
Moderator	Преглежда флагнатите коментари (<code>FlaggedForReview</code>) и ги одобрява или отхвърля.
Administrator	Дава и отнема ролята <code>Moderator</code> ; активира и деактивира профили (<code>IsActive</code>). Няма власт върху коментарите.

7.1. Персонализирани оторизационни политики (Strategy в слоя за сигурност)

Правата са описани чрез собствени двойки `IAuthorizationRequirement` + `AuthorizationHandler<T>` , регистрирани като именувани политики. Това е приложение на шаблона Strategy: всеки handler затваря в себе си едно решение „да/не“ на база ролевите claim-ове на потребителя, без да ходи до базата данни.

Политика	Дава достъп на	Бележка
<code>CanModerateComments</code>	само <code>Moderator</code>	Администраторът Е ИЗКЛЮЧЕН. Handler-ът пуска само ако потребителят е в роля <code>Moderator</code> ; самата роля <code>Administrator</code> никога не минава тази политика.
<code>CanManageModerators</code>	само <code>Administrator</code>	Дава/отнема ролята <code>Moderator</code> .
<code>CanManageAccounts</code>	само <code>Administrator</code>	Активира/деактивира профили.

Това разделение покрива направо изискването от условието: администраторът управлява *хората* (роли и активност), но не и *съдържанието*. Ако един и същ потребител е едновременно `Administrator` и `Moderator` (позволено е), той може да модерира *само* заради ролята `Moderator` .

7.2. Гейт за активност (IsActive)

Флагът `ApplicationUser.IsActive` е твърда преграда при вход и по време на сесията. Профил, деактивиран от администратора, не може да влезе; ако потребителят вече е влязъл и го деактивират, следващата защитена заявка се проверява наново спрямо записаната стойност (чрез `ActiveUserCookieValidator`), така че старо cookie не може да продължи да дава достъп. Администраторът не може да деактивира собствения си профил.

7.3. Стратегия `IRoleWorkspace`

За да показва различно табло на всяка роля, се ползва стратегията `IRoleWorkspace` с три реализации — `UserWorkspace`, `ModeratorWorkspace` и `AdministratorWorkspace`. Резолверът `RoleWorkspaceResolver` избира активното табло според ролята на влезлия потребител, с приоритет `Administrator > Moderator > User`. Всяко табло показва само действията, позволени за съответната роля, спрямо политиките от §7.1 (например таблото на администратора *не* предлага действие за модериране на коментари).

8. ML компонент — анализ на токсичност (NAS-BERT)

Анализаторът слага на коментара един от два етикета: `Clean` или `Toxic`. Абстракцията е доменният интерфейс `ICommentAnalyzer` с един-единствен метод `Task<AnalysisResult> AnalyzeAsync(string text, CancellationToken)`, който връща `AnalysisResult { ToxicityLabel Label, float Score }`.

8.1. Двете стратегии

- **`NasBertCommentAnalyzer`** — стратегията за реална употреба, която *адаптира* ML.NET модела (виж §8.2). Тя задава `Score` като вероятността текстът да е токсичен и слага `Toxic`, когато `Score ≥ ToxicityThreshold` (включително при равенство), иначе `Clean`. Прагът се чете от `ModerationOptions` чрез `IOptionsMonitor` и *не* е защит твърдо в кода.
- **`KeywordCommentAnalyzer`** — втора стратегия без модел, която сравнява текста (без значение от малки/главни букви) със списък нецензурни думи, зареден от `ProfanityListPath`. При съвпадение връща `Toxic` с оценка `1.0`, иначе `Clean` с `0.0`. Върши работа и като бърз предфилтър, и като удобен заместител в тестовете.

И двете стратегии правят *късо съединение* при празен текст или текст само от интервали — връщат `Clean` с оценка `0.0`, без изобщо да викат модела.

8.2. Object Pool за нишкова безопасност

ML.NET прави извеждането (inference) през обекта `PredictionEngine`, който **не е нишково безопасен**. Понеже Razor Pages обслужва много заявки едновременно, моделът се достъпва през `PredictionEnginePool<ModelInput, ModelOutput>` — реализация на шаблона Object Pool. `NasBertCommentAnalyzer` взема engine от пула при всяко извикване, вместо да пази един общ екземпляр. Регистрацията изглежда така:

```
services.AddPredictionEnginePool<ModelInput, ModelOutput>()
    .FromFile(modelName: "NasBertToxicity", filePath: modelPath, watchForChanges:
true);
```

Опцията `watchForChanges: true` дава *горещо презареждане*: щом `model.zip` на диска се смени, пулът зарежда новия модел без рестарт на приложението. Входът и изходът са `ModelInput { string Text }` и `ModelOutput { string PredictedLabel, float[] Score }`.

8.3. Принцип fail-safe в анализатора

Грешка в анализатора **никога** не се превръща тихомълком в `clean`. Липсващ или повреден модел, както и грешка при извеждане, се завиват в `AnalyzerUnavailableException`, която конвейерът приема за причина да флагне коментара (§6.1). Приложението може да стартира дори преди първия експорт на модела — благодарение на `watchForChanges` пулът ще го хване, щом се появи.

8.4. Обосновка на версиите: „ML.NET 2.0“ / „NET Core“ → .NET 8 + ML.NET 4.x

Условието посочва **NAS-BERT**, „**ML.NET 2.0**“ и „**NET.Core**“. Проектът е на **.NET 8 LTS** с **ML.NET 4.0.3** и **Microsoft.ML.TorchSharp 0.22.3**, ето защо:

- **Същият API.** Точно ML.NET 2.0 въведе `MulticlassClassification.Trainers.TextClassification` (NAS-BERT RoBERTa през TorchSharp). Същият този API върви без промяна и в ML.NET 4.x — алгоритъмът и интерфейсите са едни и същи. Тоест използвам по-нова версия само заради актуалност.
- **.NET 8 покрива „NET Core“.** .NET 8 е прекият наследник на .NET Core и е с дългосрочна поддръжка (LTS).
- **Защо не .NET 9.** .NET 9 е извън поддръжка от май 2026 г. и не е инсталиран, докато .NET 8 е LTS и е наличен. SDK е фиксиран на версия `8.0.125` чрез `global.json`.

9. Слой за данни (EF Core + SQL Server)

Данните се пазят с **Entity Framework Core 8** срещу **Microsoft SQL Server (LocalDB)**. Контекстът `ForumGuardDbContext` наследява

`IdentityDbContext<ApplicationUser, IdentityRole<Guid>, Guid>` и излага `DbSet` за `ForumThreads`, `Comments` и `ModerationDecisions`.

9.1. Конфигурация чрез Fluent API и конвенции за именуване

Всички имена на таблици и колони се задават *изрично* чрез Fluent API (по една конфигурация на същност), без да разчитам на конвенциите на EF. Правилата са:

- Първични ключове: `PK_[Таблица]` (напр. `PK_Comments`); външни ключове: `FK_[Дете]_[Родител]` (напр. `FK_Comments_ForumThreads`); индекси: `IX_[Таблица]_[Колони]` (напр. `IX_Comments_Status`).
- Дати и часове: тип `DATETIME2(7)` със стойност по подразбиране `SYSDATETIME()`.
- GUID първични ключове: стойност по подразбиране `NEWSEQUENTIALID()` (никога `NEWID()`), за да не се фрагментира индексът.
- Изброените типове (`Status`, `AnalysisLabel`, `Decision`) се пазят като `int`.
- Оптимистична конкурентност: колона `RowVersion` (`rowversion`) на `Comment` — ако двама модератори решат един и същ коментар едновременно, първият `SaveChangesAsync` минава, а вторият получава `DbUpdateConcurrencyException`.

- Поведение при изтриване: `Comment` → `ForumThread` е *Cascade*, а връзките към `ApplicationUser` (автор/създател/модератор) са *Restrict*, за да се запази одитната следа; профилът се деактивира чрез `IsActive`, а не чрез триене на ред.

9.2. Миграция и версионизиран SQL скрипт

Схемата се създава чрез EF Core миграцията `InitialCreate`. Освен нея има и версионизиран SQL скрипт `v1.0.0_InitialSchema.sql` със заглавен коментар, който сочи към SDD-FORUM-022 и свързаните спецификации. Скриптът създава таблиците на Identity, плюс `ForumThreads`, `Comments` и `ModerationDecisions` с описаните по-горе ключове, индекси и стойности по подразбиране.

9.3. Repository + Unit of Work + Specification

Достъпът до данни минава през три шаблона:

- **Repository** — `IRepository<T>` дава общите операции (`GetByIdAsync`, `ListAsync`, `Add`, `Update`, `Remove` и др.); конкретните репозитории (`ICommentRepository` и др.) добавят само ясно именувани заявки.
- **Unit of Work** — `IUnitOfWork.SaveChangesAsync` е единствената граница на транзакцията; репозиториите не викат `SaveChanges` сами. Смяната на статуса и записът на `ModerationDecision` се записват атомарно, наведнъж.
- **Specification** — `ISpecification<T>` събира в един обект критериите, включванията и подреждането. Конкретните спецификации (`FlaggedPendingReviewSpecification`, `PublishedCommentsByThreadSpecification`, `ActiveUsersSpecification`) се превеждат в `IQueryable<T>` от `SpecificationEvaluator` и се изпълняват на сървъра.

10. Приложени шаблони за дизайн

Проектът ползва нарочно подбрани утвърдени проектни шаблони (Gang of Four и корпоративни шаблони) [7][8], като всеки от тях решава конкретен проблем, без да усложнява нещата излишно.

Шаблон	Къде	Защо
Strategy	<code>ICommentAnalyzer</code> (две стратегии) и <code>IRoleWorkspace</code> (три стратегии).	Сменяеми алгоритми за анализа и за съставянето на ролевото табло, без купчина <code>if</code> -ове в извикващия код.
Adapter	<code>NasBertCommentAnalyzer</code> над <code>PredictionEnginePool</code> .	Привежда чуждия интерфейс на <code>ML.NET</code> към доменния <code>ICommentAnalyzer</code> и така пази домейна настрана от <code>ML</code> библиотеката.
Object Pool	<code>PredictionEnginePool<ModelInput, ModelOutput></code> .	Дава нишково-безопасно преизползване на скъпите, не-нишково-безопасни <code>PredictionEngine</code> обекти при много паралелни заявки.
Chain of Responsibility	Конвейерът за модерирание (<code>ProfanityPreFilter</code> → <code>MlToxicity</code>).	Поредица от независими проверки, всяка от които може да флагне или да пусне коментара; лесна за разширяване (<code>Open/Closed</code>).
Repository + Unit of Work + Specification	Слоят за данни в <code>ForumGuard.Infrastructure</code> .	Скрива достъпа до данни, държи една атомарна граница на запис и събира заявните критерии в обекти.
Custom Authorization Handlers (Strategy в сигурността)	<code>CanModerateComments</code> , <code>CanManageModerators</code> , <code>CanManageAccounts</code> .	Затваря всяко решение за достъп в отделен, лесен за тестване handler.
Options	<code>ModerationOptions</code> .	Връзва конфигурацията (праг, пътища, макс. дължина) към типизиран обект, който се проверява при стартиране.

Нарочно *не* са включени (за да не се преусложни проектът): `State`, `Domain Events`, `MediatR`, `CQRS`.

11. Технологичен стек

Слой / концерн	Технология (с версия)
Среда за изпълнение	.NET 8 LTS (SDK фиксиран чрез <code>global.json</code> на <code>8.0.125</code>)
Уеб рамка	ASP.NET Core Razor Pages
Данни	Entity Framework Core 8 (<code>8.0.11</code>) + Microsoft SQL Server (LocalDB)
Автентикация / оторизация	ASP.NET Core Identity (<code>8.0.11</code>)
Машинно самообучение	ML.NET <code>4.0.3</code> (<code>Microsoft.ML</code> , <code>Microsoft.Extensions.ML</code>) + <code>Microsoft.ML.TorchSharp</code> <code>0.22.3</code> (NAS-BERT TextClassification)
Тестване	NUnit <code>3.13.3</code> + Moq <code>4.20.70</code> (с <code>NUnit3TestAdapter</code> , <code>Microsoft.NET.Test.Sdk</code>)

Забележка: в стандартната конфигурация уеб приложението не ползва `libtorch-cpu` , за да остане лесно за стартиране; нативната библиотека се добавя само когато реално се обучава и зарежда NAS-BERT модел (виж раздел 14).

12. Тестване

Проектът е покрит с **209 теста** (208 минават при нормално изпълнение + 1 явен/ `[Explicit]` тест за реално обучение, който се прескача по подразбиране). Тестовите са подредени по слоеве и са вързани със спецификациите чрез NUnit категориите `[Category("SDD-FORUM-XXX")]` .

Слой	Брой тестове	Видове
Domain	48	unit — конструиране, преходи, видимост на <code>Comment</code>
Application	92	unit — услуги, CoR конвейер, handler-и, опции, ролеви табла
Infrastructure	38	10 unit (анализатори, спецификации) + 28 EF интеграционни срещу LocalDB
ML	31	unit (логика на анализатора, валидиране на корпуса) + интеграционни
Общо	209	208 изпълнявани + 1 <code>[Explicit]</code>

12.1. Подход

- **Резултатен обект** `Result<T>`. Услугите връщат явен резултат за очакваните случаи (валидиране, ненамерено, забранено), вместо да хвърлят изключения за управление на потока — така тестовете стават прости и предвидими.
- **Fakes и Моq.** Единичните тестове изолират компонентите чрез фалшиви реализации (напр. `StubCommentAnalyzer`, `ThrowingCommentAnalyzer`, `FixedClock`) и Моq, без да опират до база данни или модел.
- **Изолирани тестови бази.** EF интеграционните тестове работят срещу реален SQL Server LocalDB и проверяват схемата, записването и четенето в двете посоки, както и оптимистичната конкурентност.

Защо интеграционният тест за реално обучение е `[Explicit]`. Тестът, който наистина обучава модел и го зарежда през `PredictionEnginePool`, е маркиран едновременно с `[Explicit]`, `[Ignore]` и `[Category("RequiresTraining")]`. Това е така, защото му трябва нативната библиотека `libtorch-cpu` и бавно обучение на процесор. Затова обикновеното `dotnet test` никога не сваля `libtorch`, нито пуска бавното обучение — тестът се стартира само ръчно и нарочно.

13. Стартиране и употреба

13.1. Стартиране

Приложението се стартира с командата:

```
dotnet run --project src/ForumGuard.Web
```

При първото стартиране в режим за разработка началната инициализация прилага миграциите, създава трите роли и добавя по един готов акаунт за всяка роля, така че демонстрацията тръгва без никаква ръчна настройка:

- **Администратор:** `admin@forumguard.local` / `Admin#12345`
- **Модератор:** `moderator@forumguard.local` / `Mod#12345`
- **Потребител:** `user@forumguard.local` / `User#12345`

Внимание: това са пароли по подразбиране за разработка — сменете ги, преди да ползвате приложението реално. За демонстрация се добавят и примерна тема („Welcome to ForumGuard“) и два начални коментара — един чист и един токсичен.

13.2. Потребителски сценарии

- **Публикуване (потребител).** Влязъл активен потребител отваря тема и пише коментар. Системата го създава със статус `PendingAnalysis`, прекарва го синхронно през конвейера и веднага му слага `Published` (видим) или `FlaggedForReview` (скрит, чака преглед).
- **Опашка за модерирание (модератор).** Модераторът отваря страницата с опашката, която показва само флагнатите коментари (текст, автор, дата, етикет и оценка от анализа), и за всеки натиска „Одобри“ или „Отхвърли“. Одобреният коментар става видим; отхвърленият остава скрит завинаги. Всяко решение се записва като `ModerationDecision` (одитна следа).
- **Администриране (администратор).** Администраторът работи с две страници: за ролите на модераторите (дава/отнема `Moderator`) и за акаунтите (активира/деактивира профили). До опашката за модерирание достъп *няма*, точно заради разделението на отговорностите.

14. Как се обучава ML моделът

Обучението е **офлайн** стъпка, която прави конзолното приложение `ForumGuard.ModelTrainer` (никога по време на уеб заявка). Тренирът взема ръчно написания набор `seed-comments.csv` (колони `Text,Label`; в момента 243 реда — 117 `Clean` и 126 `Toxic`) и от него прави файла `model.zip`. Пълните стъпки (от `README.md` на тренера) са:

1. **Добавяне на нативния бекенд.** Към тренера се добавя `libtorch-cpu` (или `libtorch-cuda-12.1` за GPU):

```
dotnet add src/ForumGuard.ModelTrainer package libtorch-cpu
```
2. **Допълване на набора.** Към `seed-comments.csv` се добавят собствени коментари (по един ред на коментар, етикет точно `Clean` или `Toxic`, UTF-8, кавички по RFC-4180 при запетайи). Трябват поне 10 реда общо и поне по 2 на клас, като и двата класа присъстват.
3. **Обучение и експорт.**

```
dotnet run --project src/ForumGuard.ModelTrainer -- --data seed-comments.csv --output ../models/model.zip
```

Конвейерът е `MapValueToKey` → `TextClassification(labelColumnName: "Label", sentence1ColumnName: "Text")` → `MapKeyToValue`. Тренерът дели данните (валидационна част по подразбиране `0.2`) и извежда `MicroAccuracy`, `MacroAccuracy` и матрицата на грешките; при всяка грешка излиза с ненулев код и не оставя наполовина готов файл.
4. **Смяна на регистрацията.** В `Program.cs` извикването `AddInterimNasBertAnalyzer()` се сменя с `AddNasBertCommentAnalyzer(builder.Configuration)`, а към уеб проекта се добавят ML пакетите и `libtorch-cpu`.
5. **Насочване на пътя.** Ключът `Moderation:ModelPath` се насочва към новия `model.zip`. Благодарение на `watchForChanges: true` следващ експорт презарежда модела без рестарт.

Временна (interim) регистрация. По подразбиране приложението върви с `AddInterimNasBertAnalyzer()`, при която мястото на NAS-BERT анализатора временно се заема от `KeywordCommentAnalyzer`. Така целият проект се компилира, стартира и тества от край до край без да има обучен `model.zip` или `libtorch-cpu`. Минаването към реалния NAS-BERT модел е смяна на един ред, описана в стъпка 4.

15. Разпределение на задачите в екипа

Проектът е разработен от екип от трима студенти. Тъй като оценката се формира 2/3 от личната роля на студента и 1/3 от цялостния продукт, по-долу е описано за какво отговаря всеки от екипа.

15.1. Радослав Гурев — F112656

- Отговаряше за **ML частта** — обучението и вграждането на NAS-BERT модела: подготовка на конвейера за обучение в `ForumGuard.ModelTrainer`, изготвяне на `model.zip` и зареждането му в приложението през `PredictionEnginePool` и адаптера `NasBertCommentAnalyzer`.
- **Запозна останалите от екипа** с ML.NET и с начина на работа на модела.
- Правеше **прегледи на кода (code review)** на работата на колегите и следеше нещата между отделните слоеве да си пасват.

15.2. Алиса Спасова — F112658

- Изготви и проведе **тестовите за качество (QA)** и **ръчните тестове на самия екран** — преминаване през потоците на живо: вход в системата, публикуване на коментар, опашката за модерация (одобряване/отхвърляне) и администраторските страници.
- Работи по **част от front-end** частта (Razor страници и изгледи).
- Написа **част от документацията**.

15.3. Любослав Станков — F114428

- Разработи **част от back-end** логиката (услуги и достъп до данни в слоевете Application и Infrastructure).
- Работи и по **част от front-end** частта на форума.

16. Използване на AI асистент за консултация

По време на работата екипът ползваше **безплатен AI асистент** (Claude, модел Sonnet 4.6, през безплатния достъп на `claude.ai`) **само като помощно средство за консултация** — за изясняване на понятия, сравняване на подходи, обяснения по ML.NET и за справяне с конкретни грешки. Решенията по архитектурата, програмният код, тестовите и тази документация са дело на екипа. Асистентът беше използван така, както се ползва учебник или официална документация — като източник на обяснения и насоки, а не като автор на решението.

Наборът от коментари, върху който е обучен моделът (`seed-comments.csv`), е **написан от самия екип**, както изисква условието — „предварително тренирания върху писани от вас коментари модел“.

16.1. Примерни запитвания към асистента

По-долу са представителни въпроси, с които екипът се консултира по време на разработката:

- „Какво представлява алгоритъмът NAS-BERT и защо е по-подходящ за откриване на токсичен текст от обикновен филтър по ключови думи?“
- „В ML.NET кой `trainer` отговаря на NAS-BERT и как се настройва класификация на текст за два етикета (Clean/Toxic)?“
- „Какъв трябва да е форматът на CSV файла за обучение — какви колони и етикети очаква `TextClassification`?“
- „Има ли вграден, нишково-безопасен начин да заредим запазен ML.NET модел в ASP.NET Core, вместо да създаваме `PredictionEngine` на ръка?“
- „При предсказване излиза грешка „Could not find input column 'Label'“ — каква е причината и как се оправя?“
- „Кой проектен шаблон е удачен, за да скрием няколко различни анализатора (по ключови думи и ML) зад един общ интерфейс?“
- „Как да направим поредица от стъпки за модерация така, че всяка стъпка да може или да спре обработката, или да я подаде към следващата?“
- „Как се пише интеграционен NUnit тест за EF Core репозитори срещу LocalDB и как се изолират данните между отделните тестове?“
- „Как се прави политика за оторизация в ASP.NET Core, при която администраторът е изрично изключен от правото да модерират коментари?“

Бележка. Отговорите на асистента служеха за отправна точка и обяснение; целият код е писан, преглеждан и нагласян от екипа според конкретните нужди на проекта, а всички проектни и архитектурни решения са взети от хората в екипа.

17. Заключение

ForumGuard покрива всички функционални изисквания на задачата: форум на ASP.NET Core, в който всеки коментар се анализира автоматично за токсичност чрез предварително обучен NAS-BERT модел; чистите коментари се публикуват веднага, а флагнатите отиват в опашка за ръчен преглед от модератор, който ги одобрява или отхвърля; и три роли с ясно разделени права, при които администраторът управлява модераторите и активността на профилите, но няма власт върху коментарите.

Освен самите изисквания, проектът показва и сериозен инженерен подход: слоеста архитектура с обърнати зависимости, седем нарочно избрани проектни шаблона,

строго правило за видимостта на коментарите, принципа fail-safe при всеки проблем с анализа и широко тестово покритие (209 теста), вързано със спецификациите. Изборът на версии (.NET 8 LTS с ML.NET 4.x, който дава същия NAS-BERT API като ML.NET 2.0) е обоснован с актуалност и поддръжка, без да се отклонява от това, което условието иска.

В бъдеще проектът може да се разшири с: нови звена във веригата за модерирание (спам, дължина), класификация на повече езици, по-голям набор за обучение и метрики за качество в реално време.

18. Използвана литература

1. Xu, J., Tan, X., Luo, R., Song, K., Li, J., Qin, T., Liu, T.-Y. „NAS-BERT: Task-Agnostic and Adaptive-Size BERT Compression with Neural Architecture Search“. *Proceedings of the 27th ACM SIGKDD*, 2021. arXiv:2105.14444. [1]
2. Microsoft Learn. „Text classification“. ML.NET документация.
`learn.microsoft.com/dotnet/machine-learning` [2]
3. Jigsaw / Google. „Perspective API — Developer Documentation“. `developers.perspectiveapi.com` [3]
4. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. „BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding“. *NAACL-HLT*, 2019. arXiv:1810.04805. [4]
5. Liu, Y. и др. „RoBERTa: A Robustly Optimized BERT Pretraining Approach“. 2019. arXiv:1907.11692. [5]
6. Microsoft Learn. „Train a text classification model with ML.NET and TorchSharp / TextClassificationTrainer API“. `learn.microsoft.com/dotnet/api/microsoft.ml.torchsharp` [6]
7. Gamma, E., Helm, R., Johnson, R., Vlissides, J. „Design Patterns: Elements of Reusable Object-Oriented Software“. Addison-Wesley, 1994. [7]
8. Fowler, M. „Patterns of Enterprise Application Architecture“. Addison-Wesley, 2002. [8]
9. Microsoft Learn. „Options pattern in ASP.NET Core“ и „Configuration in ASP.NET Core“. `learn.microsoft.com/aspnet/core/fundamentals` [9]
10. Microsoft Learn. „Introduction to authorization in ASP.NET Core / Policy-based authorization“ и „Introduction to Identity on ASP.NET Core“. `learn.microsoft.com/aspnet/core/security` [10]
11. Microsoft Learn. „Entity Framework Core — Documentation“ (моделиране, миграции, конкурентност). `learn.microsoft.com/ef/core` [11]
12. Microsoft Learn. „Razor Pages in ASP.NET Core“. `learn.microsoft.com/aspnet/core/razor-pages` [12]

19. Бележка относно изходния код

Програмният код **не** е част от тази документация. Изходният код на ForumGuard се предава като отделно приложение към курсовата работа.